

SEMAPHORES

```
Activities Terminal Mar 17 21:11 manishaa@localhost:~
manishaa@localhost:~$ '# SEMAPHORES - Introduction
# A semaphore is a synchronization primitive
# that helps to control access to a shared resource
# by multiple processes/threads.
# It maintains a non-negative integer value.
# Two atomic operations are performed on semaphores:
# 1. wait() or P() -> Decrement the semaphore
# 2. signal() or V() -> Increment the semaphore
# These operations help in solving synchronization problems
# like Mutual Exclusion and Process Synchronization.

# Types of Semaphores:
# 1. Binary Semaphore (values 0 or 1)
# 2. Counting Semaphore (values 0 to n)
manishaa@localhost:~$
```

```
Activities Terminal Mar 17 21:11 manishaa@localhost:~
manishaa@localhost:~$ # BINARY SEMAPHORE - Example (Mutex)

# Used for Mutual Exclusion
# Value can be either 0 (locked) or 1 (unlocked)
# Only one process/thread can enter the critical section
# at a time.

# Example:
# semaphore mutex = 1; // initially unlocked
# wait(mutex); // P(mutex) -> acquire lock
# // Critical Section
# signal(mutex); // V(mutex) -> release lock

# Flow:
# 1. wait(mutex) -> if value > 0, decrement and enter
# else, block until it becomes -> 0
# 2. signal(mutex) -> increment and wake up one waiting process
manishaa@localhost:~$
```

```
Activities Terminal Mar 17 21:11 manishaa@localhost:~
manishaa@localhost:~$ # COUNTING SEMAPHORE - Example
# Used for managing multiple instances of a resource.
# Value can range from 0 to n.
# It allows up to n processes/threads to access
# the resource concurrently.

# Example:
# semaphore empty = n; // n empty slots in a buffer
# semaphore full = 0; // initially no full slots
# semaphore mutex = 1; // for mutual exclusion

# Producer Process:
# wait(empty); // wait for an empty slot
# wait(mutex); // enter critical section
# // produce item
# signal(mutex); // leave critical section
# signal(full); // indicate a full slot
manishaa@localhost:~$
```

```
Activities Terminal Mar 17 21:11 manishaa@localhost:~
manishaa@localhost:~$ # SEMAPHORE - Summary

# Key Points:
# 1. Semaphore is an integer value used for synchronization.
# 2. wait() or P() -> Decrement the semaphore value.
# If value < 0, the process is blocked.
# 3. signal() or V() -> Increment the semaphore value.
# If any process is waiting, wake one up.
# 4. Used to solve problems like:
# - Mutual Exclusion
# - Producer-Consumer
# - Readers-Writers
# - Dining Philosophers

# Note:
# Semaphores can be implemented using
# hardware support or software algorithms.
manishaa@localhost:~$
```